

BNI

INDONESIA NATIONAL CONFERENCE **2 0 2 6**



LOAD & CONCURRENCY TEST REPORT

700 attendees, one instant

The whole venue on the API at once — sign-in, browsing, the class rush, the expo scan and networking check-in, driven against the event's own seeded data.

TEST DATE

27 August
2026

SCENARIO

700 concurrent
attendees

DATA

856 attendees · 36 exhibitors · 4
classes

7

/

7

All correctness gates passed. No oversells, no duplicate scans, no server errors under peak concurrency.

This test compresses the busiest moments of the conference day into a single instant and drives them at the API together, checking two things at once: that nothing breaks under load, and that the guarantees hold — a class never sells 61 seats, a table never seats 9, a booth scan never double-counts.

Environment

Under test	Go API (chi · pgx/v5) + PostgreSQL, the production build, single instance
Data	Fresh database seeded with the real event data — 856 attendees, 36 exhibitors, 4 learning classes of 60 seats each, 89 networking tables of 8
Driver	scripts/load.py — 700 keep-alive HTTP clients behind ONE forwarded IP, the way a venue's WiFi NAT presents them; correctness asserted, latency measured
Machine	Apple M5 laptop, local Postgres. Absolute latency scales with hardware — run before the event on production-shaped kit.

Results by phase

1

Sign-in storm

700 attendees sign in at the same instant, all through ONE public IP — a venue is a single NAT. Real logins, bcrypt and all

PASS

700 / 700

RESULT

99

REQ/SEC

3,242_{ms}

P50

6,059_{ms}

P95

6,208_{ms}

P99

All 700 in. The rate limit follows the account being signed in to, not the building it is signed in from, so a hall behind one address is never turned away.

2

Browse storm

Every signed-in phone loads all four screens (pass, exhibitors, classes, agenda) at once — 2,800 requests

PASS

**2,800 /
2,800**

RESULT

2,674
REQ/SEC**40**_{ms}
P50**66**_{ms}
P95**76**_{ms}
P99

Every phone loaded every screen. Nothing above 80 ms.

3

Class rush

All 700 rush the four 60-seat classes the instant registration opens

PASS

**240 won •
460
refused**

RESULT

4,532
REQ/SEC**81**_{ms}
P50**87**_{ms}
P95**88**_{ms}
P99

Exactly 240 seats won, 460 turned away with a clean 409 — no class oversold, no 5xx.

4a

Expo scan storm

The booths scan the whole crowd — 700 scans across 32 booth scanners

PASS

700 / 700

RESULT

4,971
REQ/SEC**31**_{ms}
P50**59**_{ms}
P95**74**_{ms}
P99

Every scan recorded once. Zero duplicates under full concurrency.

4b

Networking check-in

700 attendees sit down at once across 89 tables of 8

PASS

700 / 700

RESULT

6,488
REQ/SEC**18**_{ms}
P50**27**_{ms}
P95**27**_{ms}
P99

Everyone seated. No table went over eight; no 5xx.

One number read honestly — the sign-in storm. p95 6.1 seconds, because 700 bcrypt password verifications hit the CPU in the same instant. That is the password hash doing its job, not a bottleneck in the app: every phase after sign-in stays in double-digit milliseconds at 2,600–6,500 req/sec. The real morning spreads arrivals across hours, so no attendee waits like this in practice; if a hard simultaneous spike is a concern, more CPU cores absorb it linearly — bcrypt scales with them.

What a load balancer changes, and what it does not. The API also runs behind nginx with several replicas. Measured on a fixed 4-CPU host, three replicas were **slower** than one (browse 691 rps against 1,271): one Go process already uses every core, so replicas add scheduling and a proxy hop rather than capacity. What they buy is the day surviving — killing a replica mid-traffic cost 0 of 120 requests. Scale replicas when you add CPU, not to go faster on the same machine.

BNI Indonesia National Conference 2026 · Accelerate — load & concurrency test report. Reproducible: `ulimit -n 4096 && BASE=<api> N=700 python3 scripts/load.py` against a freshly seeded database. System by WIT.